Embedded Systems study plan

FONTYS ICT / FIRST YEAR / COACH: IMAN

A. Rationale - Why Embedded Systems?

I am considering Embedded Systems (Intelligent Technologies) because I enjoy C and understanding how software interacts with hardware. I have started using C, ESP32, Wi-Fi and MQTT in an IoT greenhouse project, and explored alternative communication protocols. This gives me a concrete system in which to investigate sensors, timing and communication decisions. My interest in registers, peripherals, interrupts and digital logic is a direction to investigate, not evidence of mastery. Embedded engineering requires reasoning about physical behaviour, memory, timing and failure as well as writing working code. After discussing this gap with Iman, I want to build the mathematics, electronics and computer science needed to explain why a design works and when another approach is better. This makes the specialisation a considered next step, which I will test through the evidence below.

B. Learning roadmap - 12 weeks

Minimum outcome: complete at least TWO full Coursera courses, including required assessments. Budget: 54 hours over 12 weeks, averaging 4.5 hours/week. Course 1: 14 hours; Course 2: 20 hours; supporting theory and greenhouse experiments: 14 hours; evidence and reviews: 6 hours. No additional hardware is assumed.

Period	Learning objectives	Resource and reason for choosing it	Practical application	Deliverable / completion criteria
1-2	Embedded-system components, sensors/actuators and design constraints. Start functions, units, voltage, current and resistance to explain physical inputs.	**Course 1:** Introduction to the Internet of Things and Embedded Systems (UC Irvine, Ian Harris). Complete modules 1-2. Chosen to turn my existing IoT implementation experience into system-level reasoning.	**W1:** check assessment access; complete module 1. **W2:** complete module 2; draw the greenhouse sensor-to-software-to-output path. Reserve 7 h for coursework and 2 h for supporting work/evidence.	Module assessments submitted/passed as required; labelled architecture with one power, timing and reliability constraint. Explain sensor versus actuator and identify one assumption.
3-4	Hardware/software trade-offs and network layers; distinguish Wi-Fi, IP, transport and application protocols. Apply linear calibration and Ohm's law.	**Course 1:** complete modules 3-4 and all remaining requirements. Support: OpenStax linear functions and All About Circuits DC for brief worked calculations.	**W3:** module 3; calculate a two-point calibration and resistor current. **W4:** module 4; map greenhouse messages onto network layers. 7 h coursework + 2 h supporting work/evidence.	**Course 1 complete by W4:** completion record and required assessment results. Explain one hardware/software choice and correctly solve a changed calibration input without copying.
5-6	C readiness; build tools and version control. Understand how source code becomes an executable rather than treating the build as a black box.	**Course 2:** Introduction to Embedded Systems Software and Development Environments (Colorado Boulder, Alex Fosdick). Begin module 1 and the required environment setup. Chosen for assessed C and development-tool practice.	**W5:** check pointers, arrays and function basics; begin setup/module 1. **W6:** finish module 1 tasks; reproduce a clean build. 7 h coursework + 2 h C revision/evidence.	Build log, versioned source and submitted module work. Explain array bounds and pointer lifetime. **Fit review with Iman:** one appealing task, one difficulty and one open question.

B. Learning roadmap - continued

Sequence: IoT foundations > C readiness > build tools and memory > independent application. Each fortnight receives 9 hours; course assignments replace separate exercises where they overlap.

Period	Learning objectives	Resource and reason for choosing it	Practical application	Deliverable / completion criteria
7-8	Compilation, linking and Make; memory regions, storage lifetime and stack/heap distinctions. Connect program organisation to predictable embedded behaviour.	**Course 2:** module 2, then start module 3. Use GNU C manual only to repair specific pointer/storage gaps; coursework remains the priority.	**W7:** complete module 2 build work. **W8:** begin module 3; annotate where example variables and code reside. 7 h coursework + 2 h supporting work/evidence.	Submitted assessments, reproducible build and memory sketch. Predict what changes when a local variable becomes static; explain compile errors versus link errors.
9-10	Memory manipulation and validation. Apply the course concepts independently and finish the assessed project before the final review period.	**Course 2:** finish module 3 and module 4 final assessment, including required peer-review work. Retain the course toolchain for its assignments; transfer concepts to ESP32 separately.	**W9:** finish module 3 tasks. **W10:** submit final project/reviews; check completion status. 6 h coursework + 3 h supporting work/evidence. Reserve later time for revisions or outstanding grades.	**Course 2 completion target: W10.** Final submission, build instructions and boundary-case results. Explain one bug and its cause; pending grading is recorded as pending, not completed.
11-12	Consolidate reliable IoT and timing; introduce rate and accumulation as future control prerequisites. Secure the two-course completion target before extending scope.	Espressif GPTimer and MQTT specification for one experiment; 3Blue1Brown derivatives for a short maths bridge.	**W11:** resolve course revisions; choose ONE timing or MQTT outage experiment. **W12:** explain results and calculate slope/accumulated error from synthetic data. 9 h total, including final documentation and coach review.	**Minimum: TWO completed courses** with completion records and passed required work. Show an interval-error table OR numbered-message outage log. If grading delays completion, agree a dated follow-up; do not count submission alone.

C. Weekly routine

Theory and coursework - 2 h - Follow the scheduled course material; after both courses, use this time for supporting maths/electronics.

Practice and assessments - 2 h - Complete required assignments first, then the small greenhouse experiment. Course assignments count as practical work, not extra hours.

Evidence - 0.5 h - Save results, code and a short explanation of the principle, design choice and limitation. Total: 4.5 h/week.

The optional fifth hour is for catch-up. Protect the two-course minimum: reduce the final experiment if assignments take longer. Complete both courses before starting a third. PID implementation and ROS2 move beyond week 12; first develop functions, derivatives, integrals and sampling concepts.

D. Longer-term direction

Mathematics and electronics: extend algebra into trigonometry (phase and periodic signals), vectors/matrices (coordinates and coupled systems), then derivatives, integrals and differential equations in depth. Use these to analyse RC dynamics and sensor/actuator response; study logic gates and interface electrical limits.

Embedded and real-time systems: deepen architecture, registers, ADC/PWM and UART/I2C/SPI through one peripheral at a time. Then study RTOS scheduling, deadlines, task/ISR communication and race conditions. Demonstrate a measured timing budget and recovery from a failed peripheral.

Control engineering, then robotics: progress to modelling, transfer functions, stability analysis, sampling/aliasing, filtering and full PID with actuator constraints. After Linux CLI and C++ or Python basics, explore ROS2 nodes, topics, services and QoS using the official ROS2 tutorials (source). ROS2 supports communication between robotics components; it does not replace low-level control or guarantee deadlines. Start in simulation; choose an OS-compatible supported release at that time.

E. Progress evaluation

1. **Complete at least two courses:** aim for Course 1 by week 4 and Course 2 by week 10, with weeks 11-12 for corrections. Show completion records or certificates and required passing results; watching videos alone does not count.

2. **Explain:** at each fortnightly checkpoint, give a five-minute explanation without tutorial notes, covering the principle, implementation, alternative and limitation.

3. **Transfer:** produce at least three independent results: a calibration/circuit calculation, a clean build with memory explanation, and a greenhouse timing or outage experiment. Predict one changed input for each.

4. **Reproduce and diagnose:** keep inputs, versions, code and results. Re-run one example with Iman and explain one failed case and correction.

5. **Review fit and workload:** at weeks 6 and 12, use the hours log, two concrete experiences and one remaining knowledge gap to decide whether to continue Embedded Systems or investigate another direction.

Course workload and access

Verified on 29 October 2026. Course 1 requires basic computer knowledge: its headline estimate is 10 hours, while the four module estimates total 12; I budget 14. Course 2 is intermediate and lists 9 hours; I budget 20 for setup, C revision and assignments. Its first course requires no hardware purchase; a Linux/VM development environment is part of the work. These are estimates, not guaranteed completion times.

Both courses are listed for enrolment. Full assessment/certificate access may require payment; check the available enrolment or financial-aid options before starting. This plan assumes access to required assessments and does not assume free certificates. Allow for peer grading. An optional third course belongs after the two-course minimum and the foundational gaps have been reviewed.